

Multi-sprint planning and smooth replanning: An optimization model

Matteo Golfarelli, Stefano Rizzi*, Elisa Turrlicchia

DEIS – University of Bologna, Viale Risorgimento 2, Bologna, Italy

ARTICLE INFO

Article history:

Received 13 June 2012

Received in revised form 5 February 2013

Accepted 6 April 2013

Available online 23 April 2013

Keywords:

Agile methods

Scrum

Optimization models

Software engineering

Linear programming

ABSTRACT

Most agile methods divide a project into sprints (iterations), and include a sprint planning phase that is critical to ensure the project success. Several factors impact on the optimality of a sprint plan, which makes the planning problem difficult. In this paper we formalize the planning problem and propose an optimization model that, given the estimates made by the project team and a set of development constraints, produces a multi-sprint optimal plan that maximizes the business value perceived by users. To cope with the inherent flexibility and uncertainty of agile projects, our approach ensures that a baseline plan can be revised and re-optimized during project execution without disrupting it, which we call smooth replanning. The planning problem is converted into a generalized assignment problem, given a linear programming formulation, and solved using the IBM ILOG CPLEX Optimizer. Our model is validated on both real and synthetic projects. In particular, a case study on two real projects confirms the effectiveness of our approach; as to efficiency, for medium-sized problems an exact solution is found in a few minutes, while for large problems a heuristic solution that is less than 1% far from the exact one is returned in a few seconds. Finally, some smooth replanning tests investigate the trade-off between plan quality and stability.

© 2013 Elsevier Inc. All rights reserved.

1. Introduction

As empirical studies suggest, agility seems to be one of the most promising directions to overcome the problems of traditional software engineering approaches (Dybå and Dingsøyr, 2008). The basic principles stated in the Agile Manifesto by Beck et al. (2001) are followed by several agile methods, such as *Scrum* and *eXtreme Programming* (XP), that have been adopted by an increasing number of companies aimed at making the software development process faster and nimbler.

The agile philosophy promotes incremental and iterative design and implementation (Larman and Basili, 2003). One of the most popular approaches in this direction is to describe the software in terms of detailed user functionalities (*user stories*) and deliver, at each iteration (*sprint* in the Scrum terminology), the set of user stories that maximizes the utility for the users and fulfills a set of development constraints (Schwaber, 1995). Typical constraints include limiting the duration of an iteration, respecting correlations among user stories (namely, *couplings*, where two or more affine stories should be delivered together, and *precedences*, where one story should be delivered before another one is developed), and containing the non-delivery risk. Agile methods normally favor

“soft” constraints (i.e., constraints whose fulfillment is encouraged but not forced) to improve the project flexibility; however, it is recognized that some “hard” forms of constraints must necessarily be preserved (Cohn, 2004).

Clearly, as in other iterative and incremental approaches, the sprint planning phase is critical to ensure the project success (Svahnberg et al., 2010). Based on the *team awareness* principle, user story prioritization and definition of sprint boundaries are obtained by sharing and averaging the estimates given by the different team members about story complexity, utility, and correlations. For example, advancing high-utility stories could lead to an early significant result for users and encourage the team awareness; similarly, developing affine user stories within the same sprint can increase their perceived utility.

Devising a *baseline* (i.e., predictive, produced before the project start) sprint plan has a crucial role in locking resources to the project and committing due dates with customers (Herroelen and Leus, 2004). However, an essential premise of agile approaches is that no design decision is carved into stone. New requirements may arise during the project, and the plan should be flexible enough to accommodate them. Besides, projects are subject to uncertainty; it may be impossible for the project team to perfectly stick to the baseline plan for various reasons, such as underestimation of story complexity, unavailability of team members, or changing requirements, which may lead some sprints to fail, meaning that their results cannot be delivered as expected (Beck, 1999). This gives sprint planning an unstructured and evolutionary flavor that classical

* Corresponding author. Tel.: +39 051 2093542; fax: +39 051 2093540.

E-mail addresses: matteo.golfarelli@unibo.it (M. Golfarelli), stefano.rizzi@unibo.it (S. Rizzi), elisa.turricchia2@unibo.it (E. Turrlicchia).

project planning does not have, and requires the adoption of change management policies (Cao and Ramesh, 2008).

The effectiveness of a sprint planning phase mainly depends on the accuracy of estimates, and on the capability of properly taking several variables and interdependencies into account. While the first issue is mainly related to the team experience, the second one can be formulated as an optimization problem whose complexity increases with the project size. Clearly, a non-optimal solution to this problem leads to inefficiencies that easily turn into extra-costs and project delays. Though the commercial tools—such as *Mingle* (ThoughtWorks Studios, 2011) and *ScrumWorks* (Collabnet, 2011)—that support agile project management provide no support to optimal sprint planning, a number of approaches in this direction have been devised in the literature. A selection of relevant features of planning for iterative life-cycles has been proposed by Saliu and Ruhe (2005); the slightly different set of features we adopt here is aimed at providing better insight into the planning model. A classification of the most recent approaches according to these features is proposed in Table 1 (a more detailed discussion is included in Section 2).

It appears that none of the models provides comprehensive coverage of the features. Most noticeably, there is partial support to change management that has such a crucial role in agile projects. Managing change becomes critical in approaches that produce a look-ahead plan covering multiple iterations, because a significant alteration of future iterations may create problems with resource allocation and frustrate the users' expectations. The only multi-iteration approach that gives some support to change is the one by Greer and Ruhe (2004); however, a new plan is produced from scratch after each iteration without any relationship with the previously produced plan.

To fill this gap, in this paper we formalize the multi-sprint planning problem by taking into account all the features of Table 1 and propose an optimization model based on linear programming. Given the team estimates and a set of development constraints commonly adopted in this methodological context, the proposed model produces a plan that maximizes the business value perceived by users, thus relieving the team from the difficult task of quickly producing an optimal plan. With reference to Table 1, the key features of our approach can be summarized as follows:

- Scope** Ours is a multi-iteration approach that supports a single co-located and cross-functional team during a medium-to long-term planning, in a look-ahead perspective.
- Hard constraints** Precedences, that typically characterize the development process, are modeled as hard constraints. Besides, a story can be forced to be included into a given sprint.
- Soft constraints** Consistently with the agile philosophy, couplings are modeled as soft constraints by increasing the business value perceived by users if two or more affine stories are developed in the same sprint.
- Risk** We deal with the risk related to both uncertain and critical stories: an uncertain story is one whose complexity can hardly be estimated, a critical story is one that has a strong impact on the quality of the system being developed.
- Change management** We called our way of managing plan evolution in multi-iteration scenarios *smooth replanning*. The idea is to allow a baseline plan to be revised and re-optimized, if necessary, during project execution without disrupting it so as to protect the allocation of resources and preserve the milestones agreed with users.
- Planning** We provide exact solutions to small and medium problems and sub-optimal solutions (less than 1% worse than the optimal one) for more complex problems (e.g., problems with 100 user stories) in a few seconds.

We emphasize that the scope of our paper is limited to optimize the results of planning *given the team estimates*; managing the complexity of the estimation phase itself is out of scope.

Noticeably, our approach aims at preserving the key role played by the team experience and knowledge in delivering an effective plan:

- An optimal plan must be seen as an initial recommendation for the team, and it can be manually adjusted. The “best” plan may be one that also considers the personal experiences of the team members and some additional constraints that could not be formally modeled. For this reason, our model allows user stories to be explicitly forced into sprints.
- Some estimates cannot be reliably given when a plan is produced, because either requirements are not settled yet or there is not sufficient information at that time. Our model copes with this issue by distributing uncertain user stories as fairly as possible among the sprints.
- Plans are meant to be interactively and flexibly devised, used, and revised by design teams across the whole project timeline. Our smooth replanning enables the team to cope with the possible failure of a sprint (one or more user stories could not be delivered as expected), with the emergence of new requirements (one or more user stories are added), and with intrinsic changes in the development process (the development speed estimated must be adjusted).

The optimization problem is formalized as a generalized assignment problem (Martello and Toth, 1990) and is solved using the ILOG CPLEX Optimizer (IBM, 2011). To cope with the smooth replanning problem, we provide an advanced optimization model that uses a minimum perturbation strategy (Alagoz and Azizoglu, 2003) to ensure plan stability in case of changes.

The paper is organized as follows. Section 2 reviews the related literature, and Section 3 summarizes the key agile practices with particular attention to Scrum and XP. Sections 4 and 5 formalize the baseline planning and the smooth replanning problems, respectively. Section 6 presents a set of tests on both synthetic and real projects to prove efficiency and effectiveness of our approach. Finally, Section 7 draws the conclusions and sketches our future work.

2. Related literature

Agile principles gained increasing popularity over the last decade, and several approaches inspired by these principles have been proposed from both practitioners and researchers (Abrahamsson et al., 2003). Dybå and Dingsøyr (2008) propose a systematic review and comparison of different agile methods, focusing on both organizational and technical features. They also emphasize the increasing penetration of Scrum and XP practices in industries. The Scrum approach is deeply discussed by Schwaber (1995), who describes its key ideas and its life-cycle. A more pragmatic work is presented by Cohn (2004), who focuses on user stories and gives practical hints for estimating their complexity and utility.

As to utility in particular, the literature proposes different approaches to estimate the *value* of software, but a common model still lacks. For instance, in the wide context of value-based software engineering, Rönkkö et al. (2009) propose a value-decomposition matrix that combines three different software aspects (i.e., technology, design, and artifact) with three different value components (i.e., intrinsic value, externalities, and option value). Each cell of the matrix contains a specific question that should aid the analysts to correctly interpret the combination of different aspects. An

Table 1
Features of planning approaches for iterative life-cycles.

Approach	Scope	Hard constr.	Soft constr.	Risk	Change mgmt.	Planning
Greer (2004)	Multi-iter.	Preced., coupling	No	Partial	Partial	Heuristic
Denne (2004)	Multi-iter.	Preced.	No	Yes	No	Greedy
Saliu (2007)	Single-iter.	Preced., coupling	No	No	Yes	Exact
Li (2010)	Single-iter.	Preced.	No	No	Partial	Exact
Szoke (2011)	Multi-iter.	Preced., coupling	No	Yes	No	Exact
Valkenhoef (2011)	Multi-iter.	Preced.	Coupling	Yes	No	Exact
<i>Our approach</i>	<i>Multi-iter.</i>	<i>Preced., forced</i>	<i>Coupling</i>	<i>Yes</i>	<i>Yes</i>	<i>Exact</i>

alternative perspective is proposed by [Yongtae and Gwangman \(2004\)](#), who provide a model to express the software value with monetary concepts, exploiting the relationship between technology and market factors. A comprehensive description of the existing utility concepts is provided by [Khurum et al. \(2012\)](#), who distinguish four different viewpoints: financial, customer, internal business process, and innovation and learning. They propose a software value map (SVM) where each value perspective is decomposed into its main components (and sub-components) that can be used by analysts to develop a common understanding of value. Moreover, they define a practical model, based on SVM, to obtain the final utility estimation, mainly summarized in 3 steps: (1) selecting the scenario characterizing the specific project, (2) identifying the main pattern (i.e., set of value components) that best fits the selected scenario, and (3) defining a quantitative evaluation of the pattern by exploiting common components evaluations.

As to tools for agile project management, a few solutions are available. *AgileFant* ([Aalto University, 2011](#)) offers a set of basic functionalities to monitor the progress of project iterations. *Mingle* ([ThoughtWorks Studios, 2011](#)) and *ScrumWorks* ([Collabnet, 2011](#)) provide a more complete set of agile parameters to deal with user story risk, complexity, and utility. However, all these solutions lack in providing an automated solution to the sprint optimization problem.

In the broad context of project scheduling, the vast majority of research efforts over the past several years have concentrated on the development of exact or suboptimal procedures for the generation of a workable baseline schedule assuming a deterministic environment. Several models and algorithms have been proposed in the literature to this end (a survey is given by [Kolisch and Padman, 2001](#)). When a team has to schedule multiple projects at a time, as it often happens, the problem becomes even more complex and requires ad hoc approaches, as stressed by [Platje et al. \(1994\)](#). For this case, [de Boer \(1998\)](#) distinguishes two planning levels. The first level is known as *rough-cut capacity planning* and addresses the planning problem in the medium term. The second level, called *resource-constrained project scheduling*, deals with the operational (short-term) scheduling. In this two-level approach, also explored by [Gademann and Schutten \(2005\)](#), a top-down decomposition of activities into large work packages is used to reduce the medium-term planning complexity. According to the classifications of project scheduling problems proposed by [Herroelen et al. \(1997\)](#) and [Brucker et al. \(1999\)](#), our planning problem can be categorized as resource-constrained with renewable resources (i.e., manpower) available on a period-by-period basis. As in the basic PERT/CPM model, finish-start precedences with zero time lag are considered and no preemption of activities is allowed. The objective function we adopt is called a *free completion measure*, because its goal is not to minimize the project time-span, but rather to maximize the business value perceived by users.

In more recent years, the planning problem has gained increasing interest also for iterative and incremental methodologies. [Denne and Cleland-Huang \(2004\)](#) propose a software development strategy based on financial factors that can be applied in iterative contexts. An optimal sequence of requirements to deliver is

generated by maximizing along time the *net present value*, i.e., a combination of revenues, costs, and risks of each requirement. Two solution strategies are proposed: a greedy algorithm and a look-ahead approach. The first one selects the next requirement to deliver by considering the requirements with no unfulfilled precursors and maximum net present value; the second one extends the greedy approach by analyzing subsets of profitable precedence sequences. A specific model for agile methods is described by [Szoke \(2011\)](#), who proposes a conceptual model for release scheduling and provides an optimization model aimed at assigning requirements to the different iterations of a release by maximizing the overall utility delivered and considering precedences and coupling conditions. Then, he describes a branch-and-bound algorithm to solve the model incorporating risk management. Another work situated in the agile context is the one by [van Valkenhoef et al. \(2011\)](#), that is mainly focused on managing risk and uncertainty in XP projects. To this end, the authors estimate the team development speed and consider multiple sets of user stories with decreasing relevance (“must have”, “should have”, “could have” sets); the goal is to assign each user story to the most proper set by maximizing the overall utility of the sets and respecting precedences and couplings between user stories. A branch-and-bound algorithm is used to find the best solution. The limited number of sets they consider leads to a coarse-grained plan that must be refined to obtain an operative schedule (e.g., by breaking sets according to budget bounds and splitting user stories into smaller tasks). In a previous work we proposed an approach for baseline sprint planning in agile data warehouse projects ([Golfarelli et al., 2012](#)). The underlying model is simpler than the one we propose here (it does not include forced stories and it models coupling in a less expressive way), besides the replanning problem is not considered at all.

In real cases the project environment can hardly be assumed to be deterministic because of the variety of unexpected events that can occur and of the inherent imprecision of estimates, which requires policies for change management. While none of the above-mentioned works specifically deals with change management, an approach in this direction is *Evolve* ([Greer and Ruhe, 2004](#)), that is aimed at iterative and incremental contexts. A release plan includes different increments; at each stage, a set of requirements is allocated to the current and the future increments in such a way as to return the best trade-off between stakeholder priorities and development constraints (such as increment capacity, precedences, and coupling conditions). The model is formalized as a multiple knapsack problem and a genetic algorithm is used to solve it. To deal with change, *Evolve* includes a partial strategy for re-planning: at each increment, new requirements and changes in priorities and/or constraints are allowed, and a new solution is generated from scratch. In the context of Scrum planning, [Li et al. \(2010\)](#) give a knapsack formulation of an optimization model for single-iteration planning that selects the requirements maximizing the profit of the next iteration, coping with development requirements. Evolution is managed by allowing changes in parameters after each iteration and coping with their impact on the model. A new solution for next iteration is produced from scratch, by incorporating changes and additional stories. A more sophisticated approach is

bi-objective planning (Saliu and Ruhe, 2007), in which the next iteration is planned considering the impact of new requirements or changes on the existing system from either the business or the development perspective. A set of plans is generated, each reflecting a different importance of business and implementation aspects, then the optimal plan is chosen as the one that best satisfies a group of interdependencies (called *SD-couplings*) between requirements identified through impact analysis.

Though these approaches specifically address the issues related to plan uncertainty and replanning, none of them takes care of preventing the new plan from disrupting the previous one, that is one of the contributions of this work. To find some contributions in this direction we must look at the research area on *scheduling under uncertainty*, whose efforts went in two directions, namely *proactive* scheduling and *reactive* scheduling (Demeulemeester and Herroelen, 2002). Proactive (or *robust*) scheduling focuses on the development of a baseline schedule that incorporates a degree of anticipation of variability during project execution in order to protect the baseline schedule. For example, Herroelen et al. (2002) define a first aggressive schedule and then add a set of resource/time buffers that protect the critical paths. Apart from thumb rules (e.g., 50% buffer sizing rule), more sophisticated methods for sizing the buffers are discussed by Newbold (1998). Reactive scheduling refers to the schedule modifications that may have to be made during project execution. The reactive scheduling action may be based on various underlying strategies. On one hand, the reactive effort may rely on simple techniques aimed at quick schedule consistency restoration: for instance, the right-shift rule (Sadeh et al., 1993) postpones all the activities that are affected by the schedule breakdown. On the other hand, a reactive scheduling approach may involve a full rescheduling of the remaining activities. In case of rescheduling, the new schedule can differ considerably from the baseline one and this is not desirable since it

would destroy the commitments previously set, thus generating additional costs, tensions, and dissatisfaction by both customers and team members. For this reason, naïve rescheduling approaches are often based on heuristics that carry out local rearrangements of plans. Alternatively, rescheduling can use a *minimum perturbation strategy* that aims at *ex-post* stability; this strategy relies on exact or suboptimal algorithms whose objective is either the minimization of a function of the differences between the start times of each activity in the new and original schedules (El Sakkout and Wallace, 2000) or the minimization of the number of activities to be performed in different sprints (Alagoz and Azizoglu, 2003).

3. A summary of agile practices

Agile approaches are gaining an increasing attention by the research and industrial communities, and several methods based on a common set of practices have been arising from the practitioners' experience. As shown by VersionOne (2011), the Scrum method and some hybrid approaches—combining Scrum and XP practices—are preferred by companies. Since the success of a project is directly related to customer satisfaction, agile methods strive to better comply with user requests. In particular, agile principles aim at reducing the delivery time and making the development process more flexible; indeed, accelerating the time-to-market leads to overcoming the market pressure, while flexibility ensures fast reactions to both technology evolution and user requirement changes.

To achieve the aforementioned goals, agile methods propose several complementary practices. For instance, Scrum focuses on managerial and processing aspects, whereas XP defines specific techniques (such as pair programming) to make the coding phase

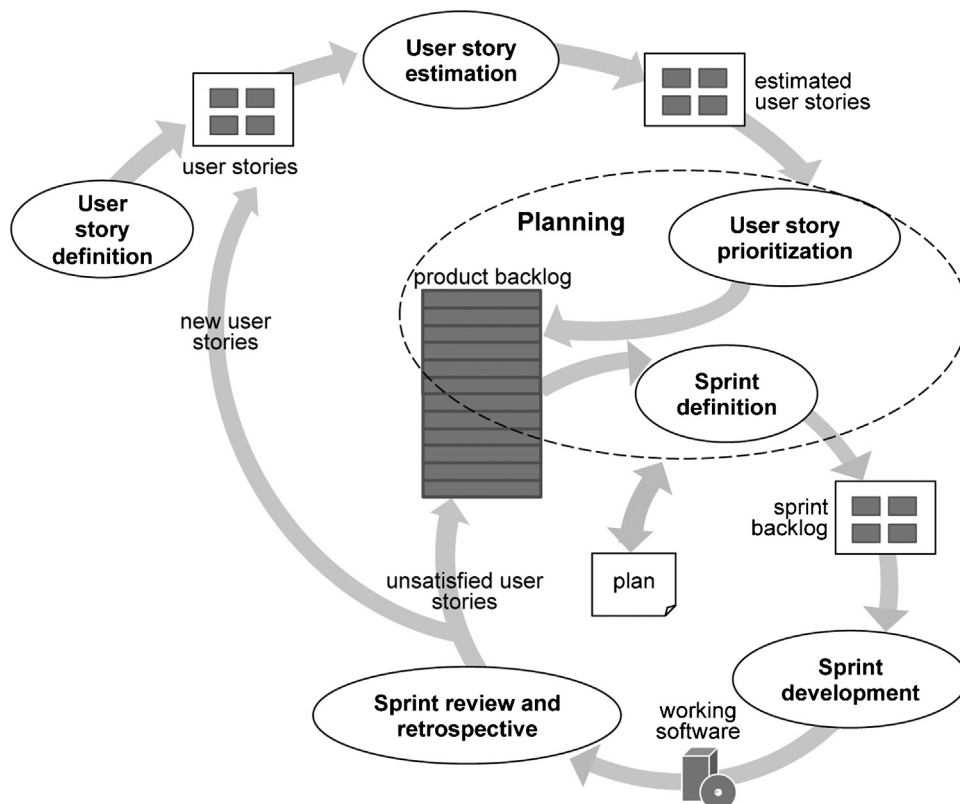


Fig. 1. Scrum life-cycle.

more efficient. Overall, the main agile practices can be summarized as follows:

- *Incremental process*: The software is broken up into smaller portions which are scheduled, developed, and integrated when completed. Each portion represents an increment in business functionality that users can validate.
- *Iteration*: The software is built in iterations, and each cycle expands the product until the project is completed. Since the process is also incremental, each iteration includes analysis, designing, coding, and testing.
- *User involvement*: Continuous interaction with users is promoted to progressively refine the project specifications, reduce inadequate requirements, and increase the trust between users and developers.
- *Team awareness*: Self-organizing teams and people collaboration are encouraged, to increase flexibility and productivity.
- *Continuous and automated testing*: Unit tests are written for each piece of code, and they are written before coding to improve software reliability; automated tests accelerate error detection.
- *Lean documentation*: Small and simple documentation is preferred to extensive specifications; thanks to continuous user involvement, up-to-date and clear documentation can be achieved.

Fig. 1 shows the typical life-cycle of a Scrum project. In the preliminary phase, the team and the user—*product owner* in the Scrum terminology—sketch the overall structure of the system and define a set of user stories. During the estimation phase, a utility and a development complexity (measured in terms of *story points*) are assigned to each user story, and possible correlations among stories are defined. Then, the team assigns a priority to each user story, also taking its risk into account. The resulting list composes the *product backlog* that must be partitioned into sprints that should be short and regular enough to guarantee a prompt feedback from users. During each sprint, the developers carry out a detailed analysis of the user stories involved, producing the *sprint backlog*: team members take charge of one or more user stories that are then designed, implemented, and tested.

After closing a sprint, the product owner verifies if the delivered stories correctly implement the requested functionalities. The approved stories are removed from the product backlog, while the remaining ones are re-prioritized. Noticeably, new requirements may arise from user feedbacks. Finally, during the *sprint retrospective* phase, the team analyzes the problems and the solutions taken so far in order to improve next iterations (e.g., the estimates made so far could be updated). Obviously, if new or modified stories arise or if estimates are relevantly changed, a new prioritization phase is executed before the next sprint is started and a new plan is produced by smoothly altering the baseline plan.

4. An optimization model for baseline planning

Our formulation of the multi-sprint planning problem is based on the static model shown in Fig. 2 that takes into account the main variables that affects user stories prioritization and sprint composition. The concepts represented are:

- *Plan*: a sequence of sprints.
- *Sprint*: the time-bound unit of iteration, typically a one- to four-week period, depending on the project complexity and risk assessment. A sprint includes a set of user stories. A maximum duration is fixed for each sprint.
- *User story*: a relatively small piece of functionality valuable for users (Cohn, 2004). It represents a light specification that can

be later detailed thanks to a continuous communication with the user; at the same time, it must be sufficiently described to estimate its development complexity. It represents a means to communicate between users and developers. In some situations the project team may want, for various reasons, to constrain some user stories (which we will call *forced*) to be included in a specific sprint; for instance, a story for developing an Android app to collect and transmit data could be forced to belong to the third sprint because only during that sprint will an Android programmer be available.

- *Utility*: the business value of a user story as perceived by the user that defines it. As stressed by Racheva et al. (2009), a detailed definition of *business value* is still missing in agile methodologies; a general and self-evident interpretation is normally assumed, related to the *earned value* defined in economics and transformable into dollar value. In practice, users normally express the value of each story using a single number, though they may implicitly take into account and combine different utility criteria. In some approaches it is only required to define an ordering for user stories (i.e., user story 1 is more useful than user story 2), but in general it can be quantified through a positive numerical score typically ranging between 10 and 100 (Nichols, 2009). For instance, a story for having a site map effectively indexed by a research engine could have utility 80, because it relevantly impacts on the site visibility on the web, while a story for showing photographs of the staff members on the site could have utility 10 because it adds small value to the site content.
- *Complexity*: the development effort for a user story measured in *story points*. Team members assign story points to each user story based on their experience and knowledge of the domain and project specificities. Story points are non-dimensional and are preferred to time/space measures to avoid subjective and incomparable estimates. Typical complexities of user stories range between 1 and 10 story points (Nichols, 2009). For instance, the indexing story mentioned above could have complexity 7, while the photograph story could have complexity 1.
- *Risk*: we consider risks related to two different characteristics of user stories. A *critical* story is one that may have a strong impact on the quality of the system delivered, so that taking a wrong solution for it dramatically affects the success of the project (e.g., a story for defining the deployment architecture that heavily impacts on performances and security). An *uncertain* story is one for which it is somehow hard to estimate the complexity due to unexpected problems that could arise (e.g., a story for feeding a database from data flows produced by a third-party company).
- *Coupling*: a correlation between two or more affine user stories. Affine stories have higher utility if they are included in the same sprint, because users better perceive the overall business value of the functionality delivered. For instance, a “zoom-out” story may have low utility on its own, but its utility may increase if delivered together with the complementary “zoom in” story. A user story can be included in several coupling groups, each characterized by an affinity: the higher the affinity, the higher the utility in jointly delivering the functionalities.
- *Precedence*: a hard constraint stating that a user story can be developed only after one or more other user stories (called *pre-conditions*) have been completed. For instance, a database can be created and populated only after its conceptual schema has been designed and documented. A conjunctive precedence (AND-type precedence) implies all pre-conditions must be completed, while a disjunctive precedence (OR-type precedence) implies at least one of the pre-conditions must be completed.¹

¹ More complex expressions, such as an OR of AND's and the like, could easily be used to model precedences, with small effects on the overall complexity of the

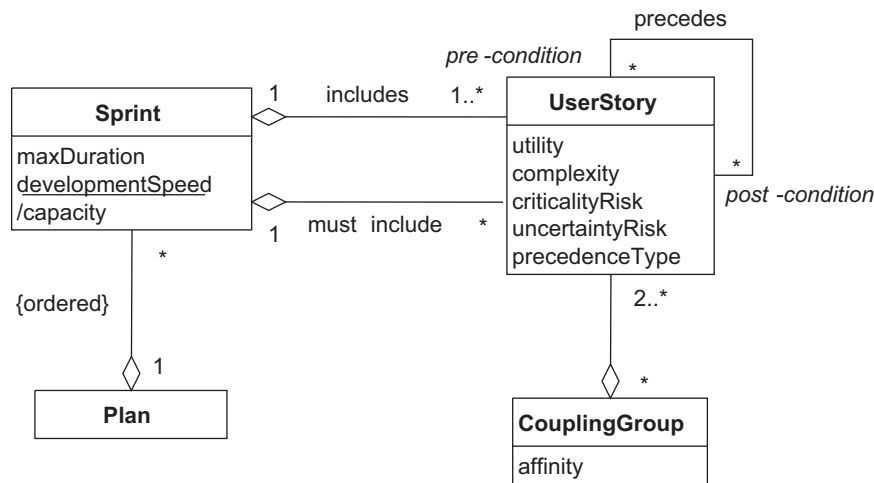


Fig. 2. The user story model as a UML class diagram (static attributes are underlined, roles are in italics).

- **Development speed:** the number of story points the team can deliver per day. It is used to convert the sprint duration into the *sprint capacity* (i.e., the maximum number of story points the team can deliver in a sprint).

To make this model applicable, reference values and ranges must be chosen for its concepts. We estimate both types of risk by associating values in the range $[1 \dots 2]$ to four classes of risk: 1 (no risk), 1.3 (low risk), 1.7 (medium risk), and 2 (high risk). Besides, the affinity range we adopt is $[0, 0.5]$, meaning that the utility of a story can be increased at most by 50%. We remark that the validity of our approach does not depend on the reference values and ranges proposed that were chosen to fit the specific features and needs of the teams we worked with. Different teams may take advantage from using finer or coarser classes and different ranges, depending on the typical precision of their estimates.

In this regard, some further considerations about utility are in order. Following the agile principles, we do not formally model the process for correctly combining different utility criteria that is left to the user experience. As a consequence we define utility as a number, which is obviously a simplification that may reduce the accuracy of estimates. On the other hand, we remark that our model can seamlessly accommodate different types of utility, meant both from the users point of view (e.g., positive impact of a story on sales and revenues, or effects on customer fidelity) or from other points of view (e.g., not degrading the overall software architecture) as long as these can be combined into a formula.

We can now list the goals an optimal baseline plan should pursue:

- #1 **Customer satisfaction.** It can be obtained by early delivering high-utility sprints. In the agile philosophy, this also increases the user awareness and trust.
- #2 **Coupling management.** Affine stories should be carried out in the same sprint to increase their utility for users. We argue that the increase in utility comes from the presence of any affine stories in the same sprint, i.e., users perceive higher utility even if only *some* of the stories in a coupling group are delivered together. In light of this, couplings can be managed by

increasing sprint utility proportionally to the number of affine stories jointly delivered.

- #3 **Risk management.** It can be achieved by in two complementary ways. On the one hand, critical user stories are advanced to reduce the risk of project failure. For instance, since a story for defining the deployment architecture heavily impacts on performances, developing this story during the last sprints could lead to realizing too late that the performances achieved are not satisfactory. If the same story had been developed earlier, the team could have taken effective correction actions in time (e.g., by implementing more efficient algorithms or choosing performance-aware design solutions). On the other hand, uncertain stories are distributed in different sprints to balance the risk that a sprint delivery is delayed. Indeed, if several uncertain stories were concentrated in the same sprint, having that sprint severely delayed due to unexpected problems would be very likely.

Besides, all constraints related to the sprint capacity, forced user stories, and inter-story precedences must obviously be met.

The problem of determining an optimal baseline plan, i.e., one that achieves these goals, can be converted into a *generalized assignment problem*, a generalization of the 0-1 multiple knapsack problem where the knapsacks are the sprints and the items are the stories.² Story points measure the weight of an item, while utility represents its value. Knapsack capacity is measured as the story points that the team can deliver given the sprint duration and the team development speed, i.e., as the sprint capacity. The objective function to be maximized is the integral of the *cumulative utility* of the project (goal #1), where the utility of each story is increased if some affine stories are included in the same sprint (goal #2) and/or if that story is critical (goal #3-i). Finally, in the formulation of the capacity constraint, the complexities of user stories are increased by their uncertainties, which discourages the inclusion of two uncertain stories in the same sprint (goal #3-ii). The generalized assignment problem is NP-hard (Fréville, 2004); the linear programming formulation we adopt is shown in the following.

optimization model. However, here we prefer to adopt a simpler form for precedences because, in our experience, it is largely adequate to accommodate the expressiveness required in practice by project teams.

² Using the terminology of operational research, the generalized assignment problem can be described as follows. Given a set of items, each with a value (that depends on the knapsack the item is assigned to) and a weight, and a set of knapsacks, each with a capacity, assign each item to one knapsack so as to maximize the total value assigned, without assigning to any knapsack a total weight greater than its capacity (Martello and Toth, 1990).

Definition 1 (Multi-sprint planning problem). Given a set of m sprints S and a set of n user stories U , let:

- $x_{ij} = 1$ iff story j is included in sprint i , 0 otherwise;
- $p_i^{\max}$ be the capacity of sprint i , measured in story points;
- u_j be the utility of story j ;
- p_j be the complexity of story j , measured in story points;
- r_j^{cr} and r_j^{un} be the criticality and uncertainty risks, respectively, of story j ;
- G be the set of coupling groups and $A_l \in G$ be a coupling group, i.e., $A_l \subset U$;
- a_l be the affinity between the stories in A_l ;
- y_{ijl} be the number of stories of A_l affine to story j and included in sprint i (y_{ijl} positive and integer);
- $F \subseteq U$ be the set of forced stories, and f_j for $j \in F$ be the sprint where story j must be placed;
- $D_j \subset U$ be the set of pre-conditions for story j ;
- $U^{AND} \cup U^{OR} \subseteq U$, where U^{AND} and U^{OR} are the subsets of stories having precedence type AND and OR, respectively.

The multi-sprint planning problem consists in determining an optimal assignment of the x_{ij} 's, i.e., in finding which stories compose each sprint in an optimal plan. Its linear programming formulation is as follows:

$$z = \max \sum_{k=1}^m cu_k, \quad \text{where } cu_k = \sum_{i=1}^k \sum_{j=1}^n u_j \left(r_j^{cr} x_{ij} + \sum_{l=1}^{|G|} a_l \frac{y_{ijl}}{|A_l| - 1} \right) \quad (1)$$

$$s.t. \sum_{j=1}^n p_j r_j^{un} x_{ij} \leq p_i^{\max} \quad \forall i \in S \quad (2)$$

$$\sum_{i=1}^m x_{ij} = 1 \quad \forall j \in U \quad (3)$$

$$x_{f,j} = 1 \quad \forall j \in F \quad (4)$$

$$\sum_{w=1}^i \sum_{v \in D_j} x_{wv} \geq x_{ij} \quad \forall i \in S, j \in U^{OR} \quad (5)$$

$$\sum_{w=1}^i \sum_{v \in D_j} x_{wv} \geq x_{ij} |D_j| \quad \forall i \in S, j \in U^{AND} \quad (6)$$

$$y_{ijl} \leq \sum_{v \in A_l, v \neq j} x_{iv} \quad \forall i \in S, j \in U, l \in G \text{ s.t. } u_j \in A_l \quad (7)$$

$$y_{ijl} \leq (|A_l| - 1) x_{ij} \quad \forall i \in S, j \in U, l \in G \text{ s.t. } u_j \in A_l \quad (8)$$

The explanation of the elements of the linear programming formulation is as follows:

- (1) The objective function z states that the optimal plan maximizes the integral of the cumulative utilities, where each cumulative utility cu_k is the sum of the utilities achieved by the first k sprints. The criticality risk r_j^{cr} increases the utility u_j of a critical story j , thus encouraging an early placement of critical stories. Couplings are managed through term $a_l y_{ijl} / (|A_l| - 1)$: for each coupling group A_l story j belongs to, the utility of story j is increased proportionally to the affinity a_l of A_l , and to the fraction of affine stories of A_l included in sprint i .
- (2) These inequalities ensure that the sum of the complexities of the stories included in each sprint i does not exceed the sprint

capacity $p_i^{\max}$. The complexity p_j of story j is increased according to the uncertainty risk r_j^{un} of that story, so as to fairly distribute uncertainty risk among the sprints.

- (3) This constraint imposes that each story is included in exactly one sprint.
- (4) This constraint correctly places each forced story $j \in F$ in sprint f_j .
- (5) These inequalities handle OR precedences by stating that, for each story j , at least one story v in D_j is placed in a sprint w before sprint i where story j is placed, that is, story v is placed before story j .
- (6) These inequalities handle AND precedences by stating that all stories in D_j are placed before each story j .
- (7) These inequalities manage couplings by counting the number y_{ijl} of stories of coupling group A_l affine to story j and carried out in sprint i . Using an inequality is necessary to accommodate the fact that, if sprint i includes stories affine to j but j is not part of i , it is $y_{ijl} = 0$ (see constraint (8)). Nonetheless, the optimizer will set the value of each y_{ijl} as high as possible since this increases the objective function.³
- (8) These inequalities state that y_{ijl} is zero if story j is not part of sprint i , otherwise it cannot be greater than the number of stories affine to it, i.e., $|A_l| - 1$.

Note that in this work we chose to model coupling correlations between stories in the form of soft constraints because hard constraints are typically avoided in agile projects. However, some planning methods in the literature take a hard approach to couplings, i.e., they can force the affine stories in a coupling group A_l to stay together in the same sprint (see Table 1). In our approach this can be indirectly achieved by replacing in U the stories that belong to A_l with a single story j_l ; as a rule of thumb, this story should be such that (i) the complexity of j_l is (at most) the sum of the complexities of the stories in A_l ; (ii) the utility of j_l is (at least) the sum of the utilities of the stories in A_l ; (iii) the uncertainty and criticality risks of j_l are the maximum of those of the stories in A_l ; and (iv) the precedences for j_l are an AND-type composition of those involving the stories in A_l .

CPLEX solves this optimization problem using a branch-and-cut approach (Caprara and Fischetti, 1997), that is, a method of combinatorial optimization for solving integer linear programming problems (i.e., linear programming problems where some or all the unknowns are restricted to integer values—the x_{ij} 's and y_{ijl} 's in our case). The method is an hybrid that dramatically improves the performance of classic branch-and-bound methods by incorporating *cutting planes*, that is, inequalities that improve the linear programming relaxation of integer linear programming problems.

Example 1. The example we report here is a simplified excerpt from the first case study presented later. The user stories considered are listed in Table 2 together with their estimations, and are allocated into 4 sprints with capacity equal to 20 story points each—except the third sprint that was given capacity 14 to model the fact that one team member is temporarily unavailable. Two precedences (from $s7$ to $s6$, from $s1$ to $s2$) and one coupling (0.3 between $s2$ and $s10$) were introduced.

The optimal baseline planning for this example is shown in Table 3; for each sprint we report its complexity (i.e., the total number of story points for the stories it includes), its uncertainty risk (i.e., the overall additional story points arising from uncertain

³ Using the auxiliary variables y_{ijl} and the related additional constraints is necessary to keep the objective function z linear. Directly counting the stories affine to story j within the objective function would make the formulation quadratic, which is recognized to be much more difficult to be optimized (Li and Sun, 2006).

Table 2
A sample of user stories from the case study.

Story Id	Story name	Utility	Complexity	Crit. risk	Uncert. risk
s1	Fee configuration	80	5	Low	Low
s2	Cash cost computation	85	2	Medium	Medium
s3	Import from IBMS	75	2	Medium	Medium
s4	Parameterization logic	30	1	Medium	Medium
s5	Amortization mask	60	2	No	No
s6	Exchange computation	60	2	Low	Low
s7	Exchange import from SAP	60	7	Low	Low
s8	Management control reporting	85	4	Medium	Medium
s9	Operational reporting	100	10	Low	Low
s10	Scenario management mask	65	3	Low	Low

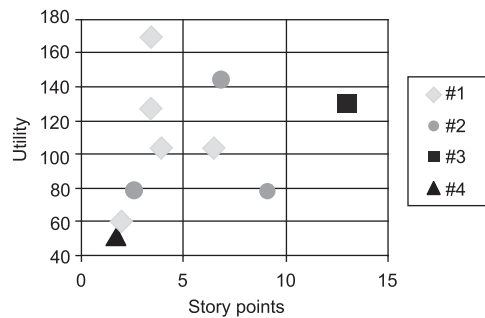


Fig. 3. Sprint composition in function of the utility and complexity of user stories for the plan in Table 3.

stories), and its cumulative utility. The integral z of the cumulative utility turns out to be 3474.5. A few remarks:

- The capacity constraint is always respected (for instance, for the first sprint, $14 + 5.2 < 20$).
- The uncertainty risk is well distributed over the first three sprints, but advanced to the first two sprints.
- The stories with higher utilities are advanced to the first sprint, also taking into account couplings and respecting the precedence from s1 to s2.
- The precedence from s7 to s6 is solved within the second sprint; these two stories have low utility and risk so they can be postponed.
- Story s9 is placed in the third sprint in spite of its high utility. In fact, if it were advanced to the first sprint it would take most of it, so it would become impossible to advance other stories with higher risk and still respect the precedences.
- The fourth sprint is not completely full; leaving some space in the last sprint is common in real projects because it allows for better managing unexpected events.

The way stories are distributed in sprints according to their utilities and complexities is illustrated in Fig. 3.

5. An optimization model for smooth replanning

As mentioned in the Introduction, the project uncertainties and the inherent flexibility of agile approaches often lead to some

disruptions from the original baseline plan. The term *predictive-reactive planning* has been coined by Vieira et al. (2003) to denote the case of a baseline plan that has been developed before the project starts and may be updated during project execution. Here we prefer to use the term *smooth replanning* to emphasize that the new plan delivered should limit as much as possible the changes made to the baseline plan; smoothness is important to protect the allocation of resources made to the projects and to preserve the milestones agreed with users.

Given the current optimal plan P (either the baseline plan or the result of a previous replanning), let U^{done} be the subset of the stories that were actually carried out at the end of sprint i , and U^{new} be the set of new stories that arose due to additional requirements. A new plan P' can be easily obtained by running again the optimization model for baseline planning on a new set of stories $U' = U - U^{done} \cup U^{new}$, and by adjusting the other variables and constraints accordingly; however, most probably, P and P' would be substantially different in the assignment of stories to sprints.

To add some smoothness to the replanning process, a proper *minimum perturbation strategy* must be adopted. Like done by Alagoz and Azizoglu (2003) we pursue a trade-off between effectiveness and stability that are respectively measured by the objective function z and by the percentage α of stories that were scheduled in corresponding sprints in P and P' . In particular, we say a new plan P' is *dominant* when for each other possible plan P'' it is either $z_{P''} < z_{P'}$ or $\alpha_{P''} < \alpha_{P'}$. Picking one dominant plan means solving a bicriteria optimization problem, which can be done in two ways. The *hierarchical approach* minimizes the secondary (i.e., less important) criterion subject to the constraint that the value of the primary (more important) criterion is kept at its optimum. The *simultaneous approach* optimizes a weighted combination of the two criteria. In this paper we adopt a hierarchical approach since we argue that maximizing utility is definitely more important in the agile context. Furthermore, the use of a complex objective function would require a parameter-tuning step to achieve the desired trade-off.

More precisely, we extend the optimization model proposed in the previous section by adding a new constraint on *suggested stories*, that is, stories whose allocation into certain sprints is desirable but not mandatory:

$$\sum_{j \in T} x_{ij} \geq \alpha |T| \quad (9)$$

Table 3
Optimal baseline planning for the stories in Table 2.

Sprint	Stories	Complexity	Uncertainty risk	Cumulative utility
#1	s1, s2, s3, s5, s10	14	5.2	565.5
#2	s6, s7, s8	13	5.5	866.0
#3	s9	10	3.0	996.0
#4	s4	1	0.7	1047.0
				$z = 3474.5$

Table 4New plan after smooth replanning with $\alpha = 0.8$.

Sprint	Stories	Complexity	Uncertainty risk	Cumulative utility
#1	s1, s3, s5	9	2.9	291.5
#2	s2, s6, s7, s10	14	5.0	721.5
#3	s9	10	3.0	851.5
#4	s4, s8	5	3.5	1047.0
				z = 2911.5

where $T \subseteq U$ is the subset of suggested stories, t_j for $j \in T$ is the sprint that should include story j , and α (stability) is the percentage of stories in T whose suggested allocation is to be respected.

This extended formulation can be used for smooth replanning by setting T to the set of stories that during the previous planning were scheduled to belong to sprints other than the current one, i.e., $T = U - U^{done}$. Noticeably, constraint (9) can also be used to deal with forced stories in a less prescriptive way; in fact, it can be seen as a relaxation of constraint (4).

Example 2. Going on with [Example 1](#), we suppose that, at the end of sprint #1, stories s2 and s10 were not completed and must be rescheduled. Smooth replanning is carried out with $T = \{s4, s6, s7, s8, s9\}$, which means that all the stories that were previously planned for sprints from #2 to #4 are suggested, while s2 and s10 can be freely allocated. By setting $\alpha = 0.8$, the team decides that at most one story in T can be disrupted ($|T| \times \alpha = 4$ stories out of 5 must be preserved). The new plan is shown in [Table 4](#); sprint #1 is in gray since it is not actually part of the current plan and it has been reported for clarity.

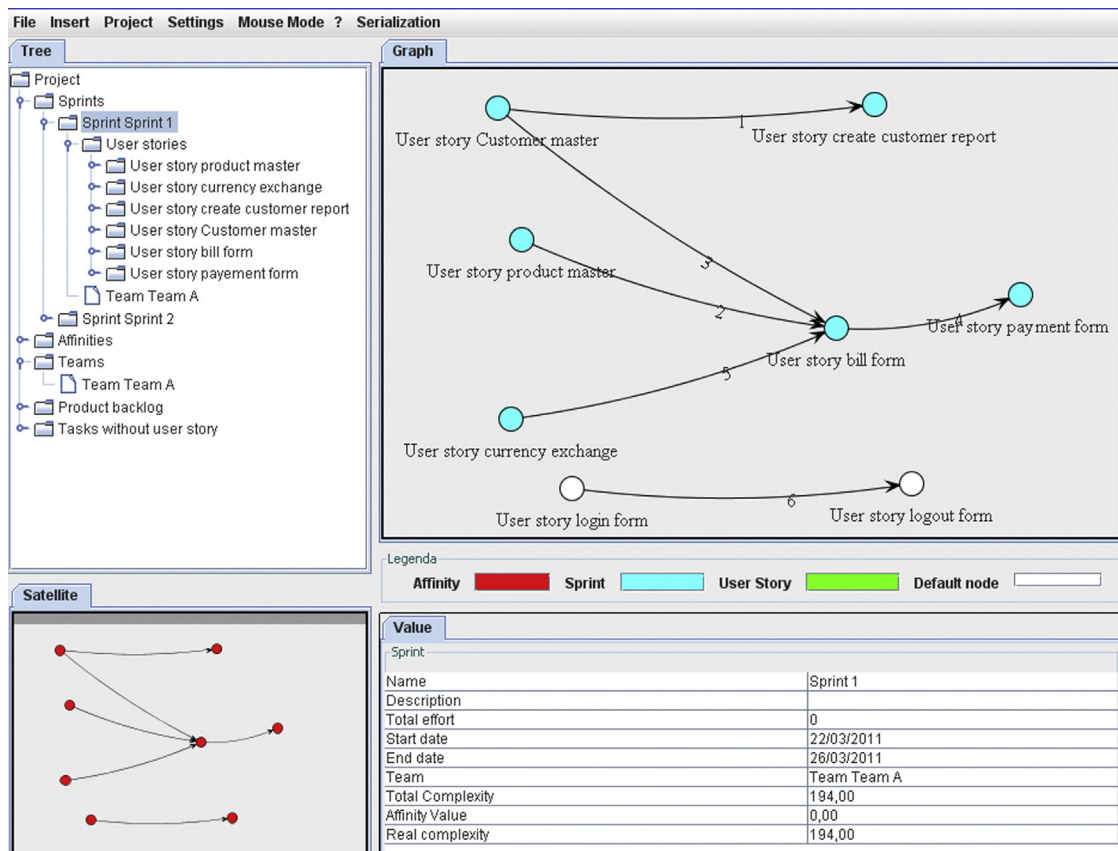
A few remarks:

- No precedence constraint is posed on s2 since s1 has been carried out in sprint #1.
- s8 has been postponed since s2 and s10 bring a higher utility and they are affine.
- The reason why s8 has been postponed instead of s6 (that has lower utility) is to leave enough space (i.e., story points) in sprint #2 to contain both s2 and s10.

6. Model validation

6.1. Effectiveness tests for baseline planning

To verify the effectiveness of our model we carried out a case study. According to the classification proposed by [Runeson and Höst \(2009\)](#), our case study can be described as *explanatory* (it aims at confirming the effectiveness of our optimization model in real contexts), *positivist* (it tests the quality of the optimal plan produced by our model), *quantitative* and *qualitative* (it quantitatively measures the quality of the optimal plan by computing the user story

**Fig. 4.** The graphical interface for planning.

gap, but it also collects a qualitative judgment by the team manager), and *flexible* (the model parameters can change during the case study). A more complete description can be given by answering the basic questions proposed by Robson (2002):

- **Objective—What to achieve?:** the case study aimed at proving the effectiveness of our approach to multi-sprint planning in the context of agile methods.
- **The case—What is studied?:** we studied two real projects with different characteristics and in different areas, namely, *Web* and *PayTV*; both projects were carried out by Italian companies that have been successfully adopting agile methods for several years.
- **Theory—Frame of reference:** the theoretical framework we adopted is the one defined by our model of planning and the related linear programming formulation.
- **Research questions—What to know?:** we studied how the optimal plan differs from the one manually produced by the project team in terms of sprint composition, risk distribution, and delivered utility.
- **Methods—How to collect data?:** for each project we collected data based on the static model of Fig. 2 during a couple of meetings made *a posteriori*, i.e., all our interactions with the team were conducted after the projects had ended. During these meetings (with an overall duration of 3 h) we asked three representatives of the team, namely the team manager, an analyst, and a developer to map onto our scale of values the estimates they had made during the estimation and prioritization phases (see Fig. 1). As to affinity and precedences, whose estimates had not been recorded using standard forms at that time, the team representatives were asked to quantify the values they implicitly had given at planning time by replicating the teamwork that is typical of the agile estimation and prioritization phases. Our role was just to explain them the exact meaning of the requested information and how to use the graphical interface shown in Fig. 4 for collecting estimates and constraints. As to utility estimation, with reference to the terminology of Khurum et al. (2012), both project teams adopted a customer perspective and limited their analysis to the *perceived value* defined in terms of relevance of the *functionalities* and their *usability*.
- **Selection strategy—Where to seek data?:** we selected two different projects to cover all the aspects involved in multi-sprint planning. *Web* is a typical agile project on web applications, with a large set of user stories and a small number of precedences; *PayTV* has a smaller number of user stories but it includes a larger set of complex precedences and couplings.

The first project—*PayTV*—was aimed at developing a data mart for a large company in the area of pay-tvs. The project had an overall duration of 8 months; it included 44 user stories and consisted of 10 sprints with an average duration of 17 days per sprint. 52 precedences (mainly of AND type) and just one coupling were involved. The project team included 4 members, but in a few cases one additional programmer was added to support the team. The development speed we used to run the optimization model was 2.43 story points per day and was empirically determined relying on historical data.

Fig. 5 compares the cumulative utilities of the optimal plan (Opt) and of the plan defined by the team (Team). The curve of the optimal plan is always higher mainly due to a better optimization of sprint composition, but also to a better handling of risk. Indeed, in the team plan some critical stories with low utility (essentially related to infrastructural needs) were advanced too much.

Fig. 6 shows the distribution of story points among the different sprints for the two plans. Remarkably, the optimal plan achieves a uniform distribution, with a light advancing of risk to the first sprints.

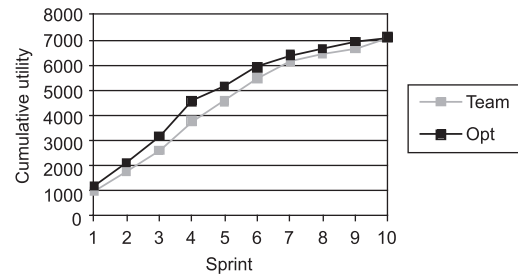


Fig. 5. Comparison of cumulative utilities for the *PayTV* case study.

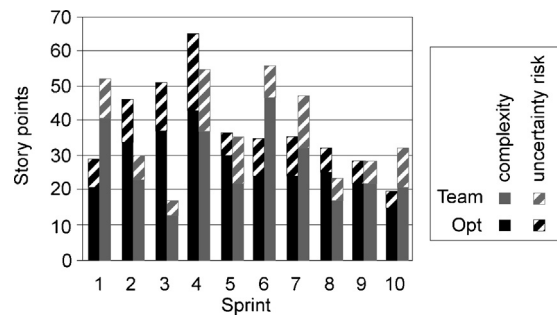


Fig. 6. Comparison of risk distributions for the *PayTV* case study.

The third comparison aims at measuring how the two plans differ in terms of sprint composition. The index we define to measure the difference between the two plans is the average of the gaps of all user stories, where the gap of a user story expresses the normalized lag of an optimally scheduled story relative to the team plan:

Definition 2 (User Story Gap). Let j be a story. Let i^{team} and i^{opt} be the sprints j belongs to in the team plan and in the optimal plan, respectively. The gap of story j is

$$gap(j) = \frac{1}{N-1} |i^{team} - i^{opt}|$$

where N is the maximum number of sprints in the two plans.

The user story gap ranges from 0 to 1, where 0 means that the story belongs to the same sprint in both plans. As shown in Fig. 7, the average gap is always lower than 0.3, denoting a good correspondence between the two plans. The main difference arises in sprints 1, 7, 8, and 10. In particular, in sprint 1, the team plan aimed at anticipating critical stories, thus exceeding the sprint capacity. The strong difference in the composition of the first sprint necessarily affected the subsequent sprints. Noticeably, both plans made a good use of coupling correlations.

In order to have a further evaluation of the optimal plan, we discussed it with the team manager after the project end. Here are the main outcomes:

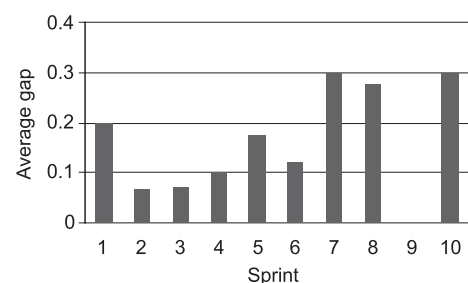


Fig. 7. Difference in sprint composition between the optimal and the team plans for the *PayTV* case study.

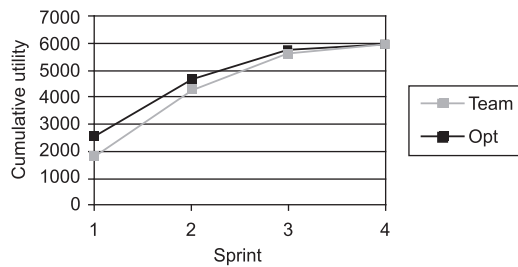


Fig. 8. Comparison of cumulative utilities for the *Web* project.

- The team spent a couple of days in defining their plan, while the optimal plan was generated in a few seconds.
- The team used to collect user story estimates using standard forms, but the level of detail required by our framework is slightly higher. This was perceived as a positive aspect since it leads to more refined estimates, thus producing a better plan. The graphical interface we provided was considered a valuable tool to support a deeper project understanding.
- The team manager recognized that his plan failed in properly distributing risks, which led to some delay in the first sprint.
- The optimal plan was judged to be feasible and realistic, showing that the elements considered in our model are sufficient to provide a good distribution of user stories.
- Most of the differences in sprint compositions were evaluated as improvements over the team plan. In particular, the team plan did not take into account the side effects of postponing some stories, thus causing the stories related to them be precedence constraints to be delayed too much.

The second project—*Web*—was aimed at developing a complex web site based on a Content Management System. *Web* is larger than *PayTV* in terms of number of user stories (105 user stories); it was organized in 4 sprints of 10 days each, so it had a shorter overall duration (40 days). This difference is due to the lower complexity of the single user stories and to a higher development speed (6 story points per day). Compared to *PayTV*, *Web* includes a small number of chain precedences (6 overall) and no couplings. The input data were collected in 4 h through an assessment with the whole project team, plus an extra session with the team manager who expressed some extra desiderata that had not emerged before:

- *Web* was the first project with a new customer; gaining its loyalty by delivering all the functionalities on time was a crucial goal of the project. Besides assigning each critical story an appropriate risk level, the team decided to anticipate some of them to the first sprint. This strategic decision goes beyond the typical development constraints; rather than modeling it by changing the risk parameters (i.e., the maximum values for r_j^{cr}), which could have undesired impacts on overall risk management, we explicitly forced the most complex user stories to the first sprint.
- Some of the requested functionalities come for free in the Content Management Systems, so they have no development complexity. Though they could be delivered in the first sprints from a technical point of view, they had better be postponed since the user cannot perceive their utility until affine stories are completed. We modeled these specific constraints using chain precedences.

After running our optimization model we compared our solution with the baseline plan devised by the project team:

- The cumulative utility of the optimal plan is higher than the one obtained by the team (see Fig. 8) and the team manager

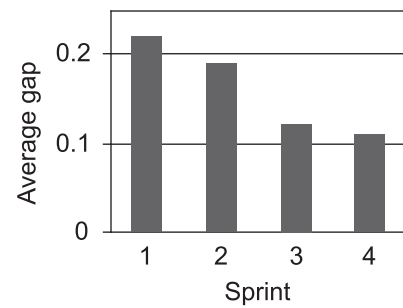


Fig. 9. Difference in sprint composition between the optimal and the team plans for the *Web* project.

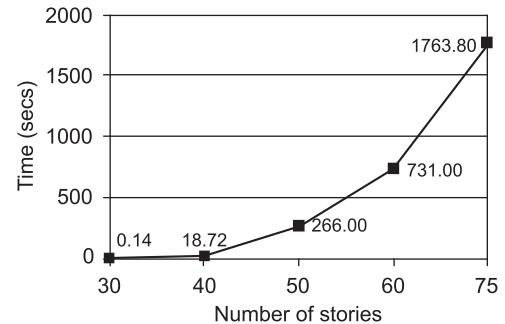


Fig. 10. Time for computing the optimal plan for projects with an increasing number of stories and no precedences.

recognized that our solution is feasible and it has a better trade-off between utility and complexity.

- The user story gap (see Fig. 9) is very low (less than 0.22 for each sprint) and is higher in the first sprint. As discussed with the team manager, two are the main motivations: (1) due to the lack of constraints and to the similar values for the utilities and complexity of user stories it was quite hard to manually define an optimal schedule; (2) the team was biased in its choices by the urge to completely deliver the first sprint, so it adopted an over-conservative solution.

Overall, from an analysis of our case study it is apparent that not only our model returns an optimal schedule, but it is also flexible and expressive enough to handle projects with different characteristics (in terms of sprint features and constraints) and it can support team-specific desiderata.

6.2. Efficiency tests for baseline planning

These tests were carried out on an Intel Core 2 Duo platform with 3 Gb of RAM, running at 3 GHz under Windows XP professional. To test the model behavior on a broad benchmark we generated a set of 58 synthetic projects; utility and complexity of the user stories were randomized in the intervals [10,100] and [1,10], respectively. The maximum sprint duration was set to 15 days, while the development speed was set to 3 story points per day (i.e., sprint capacity was 45 story points). All problems were solved using CPLEX; performances were measured in seconds.

First of all we evaluate performances in function of the total number of user stories on projects that do not include precedences. Fig. 10 reports the average time needed to compute the exact solution. As expected for a generalized assignment problem, the computation time grows non-linearly, reflecting an exponential increase in the search space.

The presence of precedences makes planning harder for the project team. To study their impact on our model, two types of

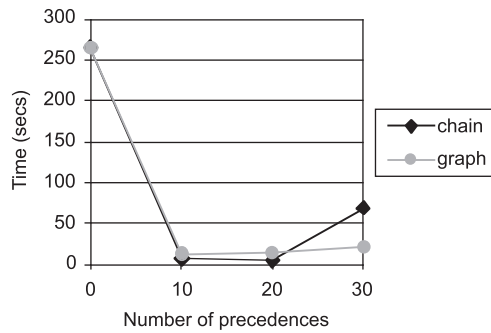


Fig. 11. Time for computing the optimal plan for projects with an increasing number of precedences and 50 stories.

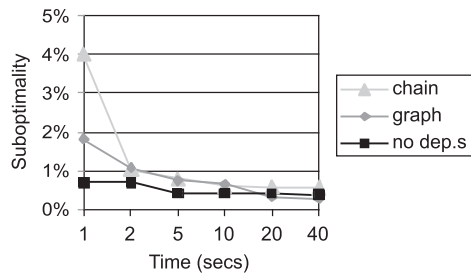


Fig. 12. Suboptimality of early solutions in function of the computation time for projects with 100 stories and 30 or no precedences.

precedences were added to our benchmark projects: (1) *chain* precedences, where each story has at most one pre-condition; and (2) *graph* precedences, where a story can have several pre-conditions. In both cases precedences were obviously acyclic. Fig. 11 shows how the computation time changes in function of the number of precedences. This figure suggests that a small number of precedences tend to reduce the computation time because precedences allow a set of unfeasible plans to be pruned, thus reducing the search space. However, when the number of precedences is high, the computation time increases again because finding a feasible plan becomes harder for the solver. Noticeably, both chain and graph precedences show similar trends.

Though the time to obtain an exact solution for very complex problems (more than 100 stories) can be too high, the time to obtain a good feasible solution is always limited. CPLEX can be configured so that it first looks for a feasible solution, then it tries to improve it until the exact one is found; at each step it returns the objective function value of the best solution found so far (i.e., an upper bound to the objective function value of the optimal solution) and a lower bound to the objective function value of the optimal solution. We measure the suboptimality at each step (i.e., how the current solution is far from the optimal one) as the ratio between the lower and the upper bounds. As shown in Fig. 12, a solution that is less

than 1% worse than the optimal one is always produced within 5 seconds. Overall, though the constraints posed in our problem do not significantly reduce the search space, CPLEX is very efficient in finding a good heuristic solution.

6.3. Effectiveness tests for smooth replanning

The effectiveness of smooth replanning can be evaluated by analyzing to what extent the previous plan is disrupted when a sprint partially fails, i.e., when it cannot deliver its expected results. To this end we considered a 50-story synthetic project and we measured the model performance when 33% of the user stories were not completed in one of its sprints. Fig. 13a shows how the value obtained for the objective function z of the new plan varies (as a percentage of the objective function value for the previous plan) in function of the sprint where the failure took place and of the stability α . As expected, due to the adoption of a cumulative objective function, the earlier the failure takes place, the worse its effects on z . Remarkably, if the failure takes place after the first sprint, the reduction in effectiveness is always less than 4% independently of the stability constraint.

Fig. 13b illustrates how the actual smoothness (meant as the percentage of stories that are not disrupted after replanning) changes with α . Noticeably, only when $\alpha=50\%$ there are cases when less stories than the maximum allowed are disrupted; in all the other cases, the smoothness fluctuations are actually due to the rounding of the number of suggested stories (e.g., given 29 suggested stories, if $\alpha=90\%$ then 2.9 stories can be disrupted; since user stories are atomic, only 2 of them can be actually moved to different sprints).

The effectiveness of smooth replanning can be also evaluated when intrinsic changes in the development process arise. In agile projects, during the review phase at the end of each sprint the development speed is estimated again, and it may be adjusted considering the feedback of past sprints and possible changes in the team composition. Then replanning is necessary to smoothly adapt the old plan to the new project parameters. An increase in speed implies an increase in the sprint capacities that may lead to an earlier placement of useful stories. Conversely, a significant speed reduction could dramatically reduce sprint capacities, forcing a late delivery of high-utility user stories. In this case, the lower the stability α , the higher the probability that a good cumulative utility is preserved at the expense of smoothness. The trade-off between quality and stability is well illustrated by Fig. 14 that shows how the objective function z of the new plan decreases with the development speed for different values of α (on the same 50-story project used in Fig. 13).

6.4. Efficiency tests for smooth replanning

Fig. 15 shows the average execution time of the smooth replanning model on our 58-project benchmark. The computation time

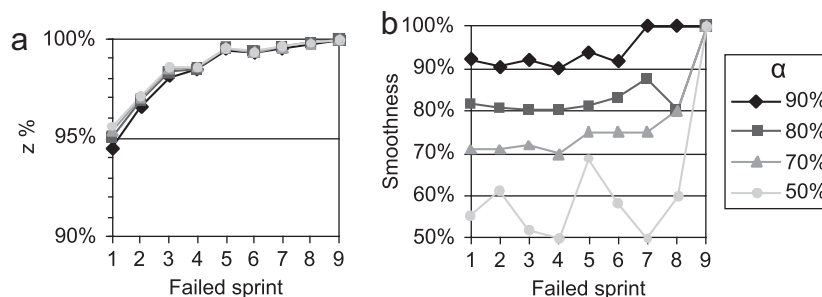


Fig. 13. Percentage objective function (a) and smoothness (b) in function of the sprint where failure took place.

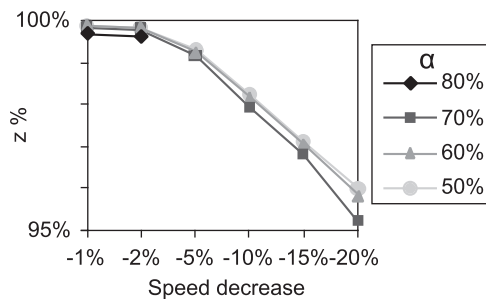


Fig. 14. Percentage objective function in function of speed decrease.

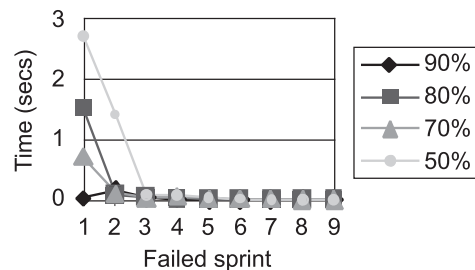


Fig. 15. Time for replanning in function of the sprint where failure took place.

is always much lower than that of baseline plans, because most of the user stories have already been assigned to sprints so that the search space is narrower.

7. Conclusions and future work

In this paper we formalized the multi-sprint planning problem and proposed a generalized assignment model to solve it. Our model was conceived for an interactive and flexible use by a design team that progressively defines the best plan or revises it during its execution; in particular, a smooth replanning process allows to effectively manage changes. Our model can be applied whenever the basic assumptions of agile methods hold, namely, definition of requirements in form of micro-functionalities (i.e., stories), capability of producing estimates of story utility, complexity, and correlation, and frequent iterations based on user feedback (which implies allocation of stories to sprints). Noticeably, our model is not geared toward a specific type of project: though in bespoke projects it can benefit from user involvement during story definition and estimation (mainly for utility assessment), it can also be effectively applied in market-driven projects where the team experience and the feedbacks received during the beta-test phase can cope with the absence of key users.

The tests we carried out show that, for medium-sized problems, an exact solution is found in a time that is fully compatible with the development process (i.e., from some seconds to a few minutes), while for large problems a heuristic solution that is less than 1% far from the exact one can be returned in a few seconds. As to effectiveness, the team managers judged the optimal plans to be feasible and realistic, and most of the differences in sprint composition were evaluated as improvements over the team plan. In smooth replanning, the trade-off between the quality and the stability of the new plan is always very good. For these reasons, we believe that our optimization module could be a very convenient and powerful add-on to the existing softwares for agile project management.

We are currently working on extending our model to better support the planning activity. Further improvements that may make the model best fit for real cases are: (1) allowing different development speeds for different sprints due to a variable team composition; (2) modeling different team capabilities (e.g., design, implement, test) so that, in each sprint, the team will be able to deliver a

different number of story points for each capability; (3) extending the model to support multiple teams working on the same project, which requires to introduce a concept of *chunks* of stories like done by Szoke (2011); (4) implementing a structured approach to utility definition and measure its impact on the accuracy of the estimates and consequently on the effectiveness of plans.

References

- Aalto University, SoberIT, 2011. Agilefant. <http://www.agilefant.org/>
- Abrahamsson, P., Warsta, J., Siponen, M.T., Ronkainen, J., 2003. New directions on agile methods: a comparative analysis. In: Proc. ICSE, pp. 244–254.
- Alagoz, O., Azizoglu, M., 2003. Rescheduling of identical parallel machines under machine eligibility constraints. *European Journal of Operational Research* 149, 523–532.
- Beck, K., 1999. Embracing change with extreme programming. *IEEE Computer* 32, 70–77.
- Beck, K., et al., 2001. Manifesto for Agile Software Development. <http://agilemanifesto.org/>
- de Boer, R., 1998. Resource-constrained multi-project management. Ph.D. Thesis. University of Twente, The Netherlands.
- Brucker, P., Drexler, A., Möhring, R.H., Neumann, K., Pesch, E., 1999. Resource-constrained project scheduling: notation, classification, models, and methods. *European Journal of Operational Research* 112, 3–41.
- Cao, L., Ramesh, B., 2008. Agile requirements engineering practices: an empirical study. *IEEE Software* 25, 60–67.
- Caprara, A., Fischetti, M., 1997. Branch-and-cut algorithms. In: Dell'Amico, M., Maffioli, F., Martello, S. (Eds.), *Annotated Bibliographies in Combinatorial Optimization*. Wiley Interscience Series in Discrete Mathematics, pp. 45–63.
- Cohn, M., 2004. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.
- Collabnet, 2011. ScrumWorks. <http://www.danube.com/>
- Demeulemeester, E., Herroelen, W., 2002. Project Scheduling—A Research Handbook. Vol. 49 of International Series in Operations Research and Management Science. Kluwer Academic Publishers, Boston.
- Denne, M., Cleland-Huang, J., 2004. *Software by Numbers*. Prentice Hall.
- Dybå, T., Dingsøyr, T., 2008. Empirical studies of agile software development: a systematic review. *Information & Software Technology* 50, 833–859.
- El Sakkout, H., Wallace, M., 2000. Probe backtrack search for minimal perturbation in dynamic scheduling. *Constraints* 5, 359–388.
- Fréville, A., 2004. The multidimensional 0–1 knapsack problem: an overview. *European Journal of Operational Research* 155, 1–21.
- Gademann, N., Schutten, M., 2005. Linear-programming-based heuristics for project capacity planning. *IIE Transactions* 37, 153–165.
- Golfarelli, M., Rizzi, S., Turricchia, E., 2012. Sprint planning optimization in agile data warehouse design. In: Proc. DaWaK, Vienna, Austria, pp. 30–41.
- Greer, D., Ruhe, G., 2004. Software release planning: an evolutionary and iterative approach. *Information & Software Technology* 46, 243–253.
- Herroelen, W., Demeulemeester, E., Reyck, B.D., 1997. A classification scheme for project scheduling problems. Technical Report. Katholieke Universiteit Leuven.
- Herroelen, W., Leus, R., 2004. Robust and reactive project scheduling: a review and classification of procedures. *International Journal of Production Research* 42, 1599–1620.
- Herroelen, W., Leus, R., Demeulemeester, E., 2002. Critical chain project scheduling: do not oversimplify. *Project Management Journal* 33, 48–60.
- IBM, 2011. IBM ILOG CPLEX Optimizer. <http://www-01.ibm.com/>
- Khurum, M., Gorschek, T., Wilson, M., 2012. The software value map—an exhaustive collection of value aspects for the development of software intensive products. *Journal of Software: Evolution and Process*, in press.
- Kolisch, R., Padman, R., 2001. An integrated survey of deterministic project scheduling. *Omega* 29, 249–272.
- Larman, C., Basili, V.R., 2003. Iterative and incremental development: a brief history. *IEEE Computer* 36, 47–56.
- Li, C., van den Akker, M., Brinkkemper, S., Diepen, G., 2010. An integrated approach for requirement selection and scheduling in software release planning. *Requir. Eng.* 15, 375–396.
- Li, D., Sun, X., 2006. *Nonlinear Integer Programming*. Springer.
- Martello, S., Toth, P., 1990. *Knapsack Problems: Algorithm and Computer Implementation*. John Wiley and Sons Ltd.
- Newbold, R.C., 1998. *Project Management in the Fast Lane—Applying the Theory of Constraints*. St. Lucie Press.
- Nichols, A., 2009. Agile Planning, Estimation and Tracking. <http://www.slideshare.net/andrewnichols/agile-planning-estimation-and-tracking>
- Platje, A., Seidel, H., Wadman, S., 1994. Project and portfolio planning cycle: project based management for multi-project challenge. *International Journal of Project Management* 12, 100–107.
- Racheva, Z., Daneva, M., Sikkil, K., 2009. Value creation by agile projects: methodology or mystery? In: Proc. PROFES, Oulu, Finland, pp. 141–155.
- Robson, C., 2002. *Real World Research*. Blackwell.
- Rönkkö, M., Frühwirth, C., Biffl, S., 2009. Integrating value and utility concepts into a value decomposition model for value-based software engineering. In: Proc. PROFES, Oulu, Finland, pp. 362–374.
- Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering* 14, 131–164.

- Sadeh, N., Otsuka, S., Schelaback, R., 1993. Predictive and reactive scheduling with the MicroBoss production scheduling and control system. In: Proc. IJCAI, Chambéry, France, pp. 293–306.
- Saliu, M.O., Ruhe, G., 2005. Supporting software release planning decisions for evolving systems. In: Proc. SEW, pp. 14–26.
- Saliu, M.O., Ruhe, G., 2007. Bi-objective release planning for evolving software systems. In: Proc. ESEC/SIGSOFT FSE, pp. 105–114.
- Schwaber, K., 1995. SCRUM development process. In: Proc. OOPSLA.
- Svahnberg, M., Gorschek, T., Feldt, R., Torkar, R., Saleem, S.B., Shafique, M.U., 2010. A systematic review on strategic release planning models. *Information & Software Technology* 52, 237–248.
- Szoke, A., 2011. Conceptual scheduling model and optimized release scheduling for agile environments. *Information & Software Technology* 53, 574–591.
- ThoughtWorks Studios, 2011. Mingle: Agile Project Management. <http://www.thoughtworks-studios.com/>
- van Valkenhoef, G., Tervonen, T., de Brock, B., Postmus, D., 2011. Quantitative release planning in extreme programming. *Information & Software Technology* 53, 1227–1235.
- VersionOne, 2011. 6th Annual State of Agile Development Survey Results. <http://www.versionone.com/>
- Vieira, G.E., Herrmann, J.W., Lin, E., 2003. Rescheduling manufacturing systems: a framework of strategies, policies, and methods. *Journal of Scheduling* 6, 39–62.
- Yongtae, P., Gwangman, P., 2004. A new method for technology valuation in monetary value: procedure and application. *Technovation* 24, 387–394.



Matteo Golfarelli received the PhD degree for his work on autonomous agents in 1998. Since 2005, he is an associate professor, teaching information systems, database systems, and data mining. He has published more than 80 papers in refereed journals and international conferences in the fields of pattern recognition, mobile robotics, multiagent systems, and business intelligence that is now his main research field. He coauthored a book on data warehouse design. He is co-chair of the DOLAP12 and of the MiproBIS Conference and member of the editorial board of the *International Journal of Data Mining, Modeling, and Management* and of the *International Journal of Knowledge-Based Organizations*. His current research

interests include distributed and semantic data warehouse systems, and sentiment analysis.



Stefano Rizzi received his PhD in 1996 from the University of Bologna, Italy. Since 2005 he is Full Professor at the University of Bologna, where he is the head of the Data Warehousing Laboratory and teaches Business Intelligence and Software Engineering. He has published more than 100 papers in refereed journals and international conferences mainly in the fields of data warehousing, pattern recognition, and mobile robotics, and a research book on data warehouse design. He joined several research projects on the above areas and has been involved in the PANDA thematic network of the European Union concerning pattern-based management systems. He is member of the steering committee of DOLAP and ER. His current research interests include data warehouse design and business intelligence, in particular multidimensional modeling, OLAP preferences, and collaborative business intelligence.



Elisa Turricchia received her degree cum laude in Computer Science from the University of Bologna, Italy, in March 2009, presenting a thesis about interoperability issues among heterogeneous data warehouse systems. Currently, she is a PhD student at the Department of Computer Science and Engineering (DISI) of Bologna. Her research interests include data warehouse design, pervasive business intelligence, and data mining techniques. In particular, her current work focuses on the study of methods for expressing and executing OLAP preference queries and for managing distributed data warehouses.